

Indian Institute of Technology Delhi

Department of Computer Science and Engineering

Accelerated Mobile Video Processing System

Real-Time Camera Pipeline Using CPU SIMD and Mobile GPU

A report submitted for

Cornerstone Project (COP7001)

Submitted by: Nitish Kumar (2025MCS2100)

Salil Gujar (2025MCS2106)

Supervisor: Prof. Kolin Paul

Repository: https://github.com/salil-123/COP7001_Project

Abstract

In this project, we built a real-time Android application that captures live camera frames, processes them on the device, and displays the transformed output continuously on screen. The main goal of the work was not just to make a working camera app, but to study how mobile hardware acceleration can improve a continuous image-processing workload. We selected Sobel edge detection as the processing task because it is mathematically clear, works on every frame, depends on local pixel neighborhoods, and has a well-defined notion of correctness.

Our system supports four execution modes: a scalar CPU baseline, a SIMD-accelerated CPU path using ARM NEON, a GPU-accelerated path using OpenGL ES compute shaders, and a hybrid mode that uses both CPU SIMD and GPU together. We also built a live performance dashboard so that the behavior of the system remains visible during execution. The dashboard shows the current mode, frame rate, processing latency, end-to-end latency, hardware activity status, and thermal proxy information.

The work was done as a two-person systems project. Our approach was incremental. We first established a correct and measurable baseline, then accelerated it step by step, and finally focused on runtime stability, switching behavior, and measurement quality. The current application runs on a physical Android phone, works fully offline, supports runtime mode switching, and demonstrates clear improvement over the scalar baseline. This report explains the problem statement from the assignment, the design choices we made, what we have actually implemented, the main engineering challenges we faced, and how the final system matches the required end-to-end mobile processing pipeline.

1. Introduction

Modern mobile phones have multiple computing resources available at the same time. A phone is not just a simple CPU-based system anymore. It includes scalar CPU execution, vector execution through SIMD instructions, and a mobile GPU that can perform large amounts of parallel work. In theory, this hardware should allow real-time workloads to run much faster. In practice, however, acceleration on mobile devices is not automatic. It requires careful system design, attention to data movement, correct synchronization, and a clear way to measure whether the acceleration is actually useful.

Our project studies this problem in the context of live camera video processing. The system captures frames from the rear camera, applies Sobel edge detection to every frame, and renders the processed output in real time. We intentionally chose a live camera pipeline rather than an offline image-processing task because the assignment emphasizes end-to-end systems work. The final result must continuously capture, process, and display frames on the device without user intervention, without external computation, and without restarting when switching between execution modes.

We approached the project as a performance engineering task. A visually correct output was necessary, but it was not enough. The more important question was how to demonstrate measurable acceleration clearly and honestly. Because of that, we treated the baseline implementation as a reference system and built all optimized modes around it. We also added live measurement tools directly into the app so that an observer can understand what the hardware is doing while the system is running.

2. Problem Statement

The problem statement of the project can be expressed in a simple way:

Design and build an Android application that takes live camera frames, processes them continuously on the device, and demonstrates measurable performance improvement using CPU SIMD, mobile GPU execution, and a hybrid combination of both.

The assignment also places clear conditions on the system:

- The application must work as a live camera pipeline.
- Processing must happen on every frame.
- The output must be shown on screen in real time.
- The work must happen fully on the mobile device.
- The application must support multiple execution modes at runtime.
- The outputs of the different modes must remain functionally equivalent within acceptable tolerance.
- The performance improvement over baseline must be visible and measurable.
- A live dashboard is a mandatory part of the final system.

This means the project is not only about writing a faster filter. It is about building a complete end-to-end system where correctness, speed, responsiveness, and observability all matter together.

2.1 What the Assignment Specifically Expects

After reading the assignment document carefully, we understood that the final system must satisfy five central ideas:

1. It must be a **live camera pipeline**, not an offline image-processing demo.
2. It must support **four execution modes**: baseline, SIMD, GPU, and hybrid.
3. It must show **measurable acceleration relative to baseline**.
4. It must include a **real-time dashboard** that is visible while the app is running.
5. It must run **stably on a physical Android phone** without manual intervention during normal use.

We kept these five points in mind throughout the project. They shaped not only the implementation but also the way we measured and cleaned up the final system.

3. Why We Chose Sobel Edge Detection

We chose Sobel edge detection because it is a good match for the assignment and for the hardware-acceleration study we wanted to perform.

First, Sobel is a local neighborhood operation. Each output pixel depends on a small three-by-three neighborhood around the input pixel. This makes it computationally meaningful and more interesting than a trivial per-pixel color operation.

Second, Sobel has a clear correctness reference. If we know the input pixels, then the expected output can be calculated directly. This is important because once we start using SIMD and GPU execution, we need a reliable way to compare accelerated outputs against the scalar baseline.

Third, Sobel is heavy enough to show the difference between processing approaches. A simple transformation might not reveal much about acceleration, but Sobel includes repeated neighborhood access, weighted accumulation, and per-pixel output generation across the full frame.

Finally, the visual output of Sobel is easy to inspect. It highlights edges in the camera scene, so if something goes wrong, the error often becomes visible immediately. This helped us during development and debugging.

4. Overall System Approach

We followed a layered and incremental approach rather than trying to implement everything at once. As a group of two people, this was important because it allowed us to divide the work into understandable stages and verify each stage before moving ahead.

Our approach had the following major ideas:

1. Build a working scalar baseline first.
2. Keep the baseline as the correctness reference for all future modes.
3. Add acceleration one layer at a time: SIMD first, then GPU, then hybrid.
4. Keep the system observable at every stage through a dashboard.
5. Measure the real end-to-end behavior of the app, not just isolated kernels.
6. Fix runtime issues such as switching stability, resource cleanup, and misleading measurements as part of the final system design.

This approach helped us avoid a common problem in acceleration projects, where optimization happens too early and makes the system difficult to reason about. Instead, we kept asking two questions at every stage:

- Is the output still correct?

- Is the acceleration actually useful in the complete live pipeline?

5. System Architecture

At a high level, the architecture of our application is:

Live Camera Input → Native Frame Processing → Display Output → Live Dashboard

The camera subsystem captures frames using the Android Camera2 API. These frames arrive in YUV format through an `ImageReader`. The image-processing work is performed in native C++ through JNI, because that is where we need direct access to NEON intrinsics and GPU compute setup. The processed data is then converted into a displayable bitmap and shown on screen in the Android UI. On top of the camera output, we render a compact dashboard that presents the current performance state of the system.

The application currently supports:

- Baseline mode
- SIMD mode
- GPU mode
- Hybrid mode
- Runtime switching between modes
- Runtime switching between 720p and 1080p processing
- A live dashboard with system and acceleration indicators

Although the UI is visible, we did not treat the project as a UI design exercise. The UI is present only to support continuous display and live measurement. The main engineering work is in the camera pipeline, native processing, hardware acceleration, and runtime stability.

6. How Our Current Work Matches the Assignment

At this stage, our implementation matches the assignment in the following direct way:

- **Live camera input:** the app captures frames continuously from the rear camera on a physical Android phone.
- **On-device processing:** all processing is done locally on the phone using native C++ and OpenGL ES compute shaders.
- **Continuous display:** processed frames are shown live on screen without offline pre-processing.
- **Multiple modes:** baseline, SIMD, GPU, and hybrid modes are all available in the same application.
- **Runtime switching:** the user can switch modes while the app is running without restarting the app.

- **Dashboard:** the app shows mode, FPS, processing latency, end-to-end latency, SIMD activity, GPU activity, and thermal proxy information.
- **Correctness checking:** SIMD and GPU outputs are verified against the scalar baseline within acceptable tolerance.
- **Visible performance gain:** the accelerated modes are visibly faster than baseline, especially at 720p and also clearly at 1080p compared to the scalar path.

So the app is not only conceptually aligned with the assignment. It already implements the core execution model that the assignment asks for.

7. Phase 1: Building the Baseline Camera Pipeline

7.1 Project Setup

We began by creating the Android project and enabling native C++ integration through the NDK and JNI. This was necessary because the accelerated parts of the assignment require direct access to lower-level hardware-oriented programming techniques that are not available through normal Kotlin code alone.

The first goal was simply to confirm that the Android application, the native toolchain, and the Kotlin-to-C++ bridge were working correctly. Once that base layer was established, we moved to the camera side.

7.2 Camera Input and Frame Flow

We used the rear camera through the Camera2 API and configured a continuous preview pipeline. The camera delivers frames in YUV_420_888 format. These frames are received on a background thread through an `ImageReader`. Using a background thread is important because camera callbacks and processing work should not block the Android UI thread.

At this stage, our goal was not acceleration. The goal was to get a stable and continuous frame flow from the camera to our processing logic and then to the screen.

7.3 YUV to RGBA Conversion

The camera does not directly provide the processed frames in a format that can be used easily by our edge detector or by the display layer. Therefore, we implemented a native YUV-to-RGBA conversion step. This conversion reads the Y, U, and V planes carefully using their real memory layout information, including row stride and pixel stride.

This part was important for two reasons. First, it made the pixel data usable for the rest of the pipeline. Second, it gave us an initial place to measure cost inside the frame-processing path.

7.4 Scalar Baseline Sobel Filter

Once the frame format was ready, we implemented the scalar CPU version of Sobel edge detection. This baseline version processes each frame using ordinary nested loops and applies the Sobel operator on every pixel. Border handling is done carefully so that the filter remains valid on the edges of the image as well.

This scalar implementation became the ground-truth reference for the entire project. Every later optimization was compared against it for correctness. This was one of the most important design choices we made, because without a trusted reference the accelerated outputs would be difficult to validate.

7.5 Displaying the Output

After processing, we rotate and present the result on screen. Since the phone is held in portrait orientation while the camera frame is naturally landscape, we needed a display path that keeps the output visually correct for live viewing. The goal was not only to confirm that the filter works, but to make sure that the whole system satisfies the end-to-end requirement:

$$\text{capture} \rightarrow \text{process} \rightarrow \text{display}$$

7.6 Initial Dashboard

We added a basic live dashboard as soon as the baseline mode was working. This was done early on purpose. We did not want the dashboard to be an afterthought. Since the assignment explicitly treats the dashboard as a core deliverable, we wanted it to grow together with the system rather than being attached at the end.

At the baseline stage, the dashboard started with mode information, frame rate, and latency readings. Later, we expanded it as the acceleration modes became available.

8. Phase 2: SIMD Acceleration on the CPU

8.1 Why SIMD Was the Next Step

After building the baseline, the most natural next step was CPU SIMD acceleration. This kept the work on the CPU side, avoided the complexity of GPU setup in the early stages, and allowed us to validate the idea of hardware-aware acceleration using a controlled method.

On our target devices, the relevant SIMD instruction set is ARM NEON. NEON works with vector registers, which means that instead of processing one small pixel value at a time, we can process many values in parallel.

8.2 Warm-Up with a Simpler SIMD Operation

Before applying SIMD to the Sobel filter directly, we first wrote and tested a simpler NEON-based operation. This allowed us to confirm that the build configuration was correct, the intrinsics were working, and our understanding of data layout was sound.

This warm-up stage was useful because Sobel is more complex than a basic per-pixel operation. By starting simpler, we reduced the chance of making difficult-to-debug mistakes later.

8.3 NEON Sobel Design

For the actual SIMD Sobel filter, we designed the loop so that it processes sixteen pixel positions at a time for each channel group. The reason is hardware-based: NEON uses 128-bit vector registers, and for 8-bit values this maps naturally to sixteen lanes per vector. So the design choice was not arbitrary. It follows directly from the vector width of the architecture.

The optimized SIMD path reorganizes the neighborhood operations in a vector-friendly way, computes the horizontal and vertical gradients, and writes back the result in the same general format as the baseline. Any remainder at the edges is handled carefully to keep the full image correct.

8.4 Correctness Verification for SIMD

We did not enable SIMD mode based only on speed. We also implemented correctness verification relative to the baseline. The app compares the SIMD output against the scalar reference and checks whether the difference stays within a very small tolerance. This matters because small arithmetic differences can appear when integer vector operations are used, but the output must still remain functionally equivalent.

This step gave us confidence that the speedup was real and not coming from a broken or simplified implementation.

8.5 What SIMD Gave Us

The SIMD mode reduced the cost of the Sobel stage significantly compared to the scalar baseline. More importantly, it demonstrated that the app was now able to exploit CPU vector hardware in a visible and measurable way. This was the first point at which the app began to satisfy the assignment’s acceleration requirement in a meaningful form.

From our current phone runs, SIMD is the strongest mode in terms of pure Sobel latency. At 720p, baseline Sobel is roughly in the high-thirty millisecond range while SIMD Sobel is around five milliseconds. At 1080p, baseline Sobel goes above eighty milliseconds while SIMD remains around ten milliseconds. This is a very clear and visible acceleration over scalar execution.

9. Phase 3: GPU Acceleration

9.1 Why GPU Acceleration Was Harder

GPU acceleration was more complex than SIMD for several reasons. Unlike the NEON path, which remains inside the CPU and main memory domain, the GPU path introduces a new execution device, a separate programming model, and explicit synchronization concerns. It is not enough to write a shader and assume that it will be faster. Data must be uploaded, compute work must be dispatched, the result must be made visible, and if the result needs to return to the CPU side then readback cost also becomes important.

This means that GPU acceleration in a real-time mobile pipeline is a system design problem, not only a kernel-writing problem.

9.2 Headless Compute Context

To support GPU processing, we created a headless OpenGL ES compute environment through EGL. This allows compute shaders to run without requiring a traditional on-screen graphics pipeline. This was important because our use case is compute-oriented, not a standard 3D rendering problem.

We verified the setup with a simple pass-through operation before trusting it for Sobel. This was similar in spirit to our SIMD warm-up: verify the mechanism first, then move to the real workload.

9.3 GPU Sobel Design

The GPU Sobel path uses a compute shader where each invocation is responsible for one output pixel. The shader reads the local neighborhood, computes the Sobel response, and writes the result into GPU-managed output storage. Over time, we improved this path by moving the input to a texture-based read path, because image-oriented access patterns benefit from texture cache behavior more than a purely linear storage-buffer access pattern.

We also benchmarked multiple workgroup configurations so that the active GPU variant is selected based on measured behavior on the device rather than fixed assumptions.

9.4 Correctness Verification for GPU

Like the SIMD path, the GPU path is compared against the scalar baseline. Here the numerical tolerance is slightly larger, because GPU execution can involve small floating-point or representation differences depending on the exact data path and shader implementation. Even then, the output must remain functionally equivalent.

9.5 What We Learned About GPU Performance

One important lesson from this phase was that GPU acceleration is not automatically better than SIMD. For a relatively local filter like Sobel, the compute work may be fast, but the total pipeline cost can still be affected by texture upload, dispatch overhead, synchronization, and readback. This is why we later optimized the GPU path further by reducing immediate same-frame readback stalls through a pipelined output approach.

This helped us understand the difference between a theoretically parallel kernel and a practically efficient mobile pipeline.

Our current results show that GPU mode is clearly better than baseline, but it is still not always better than SIMD. This is an honest and important result. It shows that the assignment is really about systems engineering and measured acceleration, not about assuming that the GPU must always win.

10. Phase 4: Hybrid CPU-GPU Execution

10.1 Why Hybrid Mode Matters

The assignment requires not only separate SIMD and GPU modes, but also a coordinated hybrid mode. We treated this as an opportunity to show real heterogeneous computing rather than merely multiple isolated implementations.

The idea of hybrid mode is simple: instead of giving the whole frame to one device, split the work across CPU SIMD and GPU so that both hardware resources contribute to the same frame.

10.2 Hybrid Split Strategy

Our hybrid design splits the frame horizontally. One part is processed by the SIMD path and the other part is processed by the GPU path. Once both halves are finished, the outputs are combined into the final displayed image.

At first, a static split is the simplest choice, but we improved the design by making the split adaptive. The app observes how long the CPU half and GPU half take, estimates their per-row cost, and updates the next split so that both sides are more likely to finish around the same time.

This adaptive strategy is important because the CPU and GPU do not necessarily scale in the same way, and the faster side should be assigned more work if the system is to use both resources well.

10.3 Why Hybrid Mode Is Useful

Hybrid mode is useful for two reasons. First, it satisfies the assignment's requirement for coordinated heterogeneous execution. Second, it demonstrates a systems-level idea:

performance is not only about making a single path faster, but also about balancing work across multiple devices.

In our current app, hybrid mode works correctly and both resources become active at the same time. We also expose the split ratio on the dashboard so that the contribution of each side remains visible during runtime, which matches the assignment’s emphasis on resource-aware observability.

11. Runtime Switching and Stability Work

11.1 Need for Reliable Runtime Switching

The assignment requires switching between modes at runtime without restarting the application. This sounds simple at the UI level, but in practice it affects the camera pipeline, the native processing path, and the GPU resource lifecycle.

During development, we found that careless runtime switching could create stale callbacks, old frame readers, or camera-session races. These problems become even more visible when the app also supports resolution changes.

11.2 How We Stabilized Switching

We improved runtime behavior in several ways:

- We tied camera restart behavior to real resolution changes and surface availability rather than ordinary UI recomposition.
- We added protection against stale camera open and session callbacks.
- We dropped old frames during resolution changes so that ghost frames from a previous resolution do not pollute the pipeline.
- We reset measurement state when a resolution change occurs so that old and new frame sizes are not mixed in the same statistics.

These changes were important because a fast app that hangs or becomes inconsistent when switching modes would not satisfy the end-to-end system requirement.

12. Performance Dashboard Design

12.1 Why the Dashboard Is Central

The assignment clearly states that the dashboard is a mandatory deliverable. We treated it as a live explanation layer for the system. The purpose of the dashboard is not decoration. It exists so that anyone watching the app can understand what is currently running and how the hardware resources are being used.

12.2 What the Dashboard Shows

The current dashboard includes the following kinds of information:

- Current execution mode
- Live frame rate
- End-to-end latency
- Sobel processing latency
- SIMD activity state
- GPU activity state
- Thermal proxy
- Jitter count
- Speedup over baseline
- Hybrid split ratio when hybrid mode is active

We intentionally simplified the dashboard by removing several low-level debug details that were useful during development but not necessary for the final demonstration. This made the dashboard closer to the assignment requirement and easier to understand during a live demo.

This final dashboard design is important for assignment alignment. The assignment does not ask for every internal micro-timing to be shown. It asks for a dashboard that helps the observer understand mode, latency, FPS, and how computation is mapped to SIMD and GPU resources. That is the version we now keep visible in the app.

12.3 Fixing Misleading Speedup Behavior

One subtle problem we discovered was that speedup can become misleading if it is computed from unstable frame samples. For example, if the system stores one old SIMD measurement and then keeps updating baseline every frame after switching back, the ratio fluctuates in a way that is mathematically valid but not fair as a comparison.

To fix this, we changed the speedup logic so that it uses stabilized per-mode averages rather than single-frame snapshots. We also ignore a few warm-up frames after switching modes and reset the averages when resolution changes. This makes the reported speedup more meaningful as a performance comparison.

13. Main Engineering Challenges We Faced

This project looked simple from far away, but in practice it involved several non-trivial engineering challenges.

13.1 Challenge 1: Keeping the Whole Pipeline Real-Time

It is possible to optimize one stage while ignoring the rest of the pipeline, but that would not satisfy the assignment. We had to keep asking whether the app was behaving well as a complete end-to-end system rather than just whether one kernel became faster.

13.2 Challenge 2: Correctness Across Modes

Every accelerated mode must remain equivalent to baseline. This required careful comparison logic and discipline in treating the scalar path as the source of truth.

13.3 Challenge 3: GPU Overhead

The GPU path was not just about shader speed. Transfer and synchronization overhead mattered a lot. This taught us that acceleration must be evaluated in system context, not in isolation.

13.4 Challenge 4: Mode and Resolution Switching

Runtime switching exposed lifecycle issues that would not appear in a single-mode pipeline. We had to fix stale resource reuse, synchronization races, and measurement contamination across different resolutions.

13.5 Challenge 5: Measurement Quality

Performance numbers can easily become noisy or misleading. A major part of our effort went into making the measurements honest enough to support real comparison.

14. Current State of the System

At the current stage, the application has the main features expected from the assignment:

- Real-time camera input on a physical Android device
- Continuous on-device frame processing
- Live visual output
- Scalar baseline mode
- SIMD-accelerated mode
- GPU-accelerated mode
- Hybrid mode
- Runtime mode switching
- Runtime resolution switching
- Correctness verification relative to baseline
- Live performance dashboard
- Explicit SIMD and GPU activity indicators

- Thermal-aware hybrid split control

The project is therefore no longer just a partial prototype. It functions as a coherent accelerated mobile video-processing system.

14.1 Observed Runtime Behavior

From our current device screenshots and testing:

- At **1280×720**, baseline runs around twenty FPS, while SIMD and GPU reach around thirty FPS, and hybrid also remains interactive.
- At **1920×1080**, baseline becomes much slower, while SIMD, GPU, and hybrid remain far better than the scalar mode.
- SIMD is currently the best mode in Sobel latency on our tested device.
- GPU is still clearly useful because it beats baseline and satisfies the GPU execution requirement, even though it does not yet outperform SIMD.
- Hybrid mode works correctly, shows both hardware resources as active, and adapts its split ratio during runtime.

These observations are important because they show that the assignment requirement of *measurable improvement over baseline* is already satisfied by the implemented system.

14.2 Representative Screenshot Results and Metric Computation

To keep the report concise, we include only a few representative screenshots from the final app instead of all captured screenshots. The four images below summarize the most important 720p results: baseline, SIMD, GPU, and hybrid. They show the exact dashboard that is implemented in the codebase and therefore serve as representative evidence of the runtime behavior that the application actually measures.

At 720p, the baseline screenshot shows about **18 FPS**, **54.51 ms** end-to-end latency, and **41.82 ms** Sobel latency. The SIMD screenshot shows about **30 FPS**, **19.53 ms** end-to-end latency, and **6.66 ms** Sobel latency. The GPU screenshot also reaches about **30 FPS** with **18.91 ms** end-to-end latency and **7.33 ms** Sobel latency. The hybrid screenshot reaches about **30 FPS**, reports **24.44 ms** end-to-end latency and **8.35 ms** Sobel latency, and shows a live split of **55/45** between CPU SIMD and GPU. At 1080p, the screenshots show the same trend even more strongly: baseline Sobel latency is around **92.51 ms**, whereas SIMD remains around **14.42 ms**, which explains why the scalar path becomes much less usable at the higher resolution.

The dashboard metrics are computed directly from the live pipeline implemented in `MainActivity.kt`. The **FPS** value is estimated from a short history of recent frame-arrival timestamps recorded inside the camera callback. The code keeps up to thirty recent timestamps and computes FPS from the total time span covered by those samples. The **end-to-end latency** is measured as the time from the moment a frame is acquired

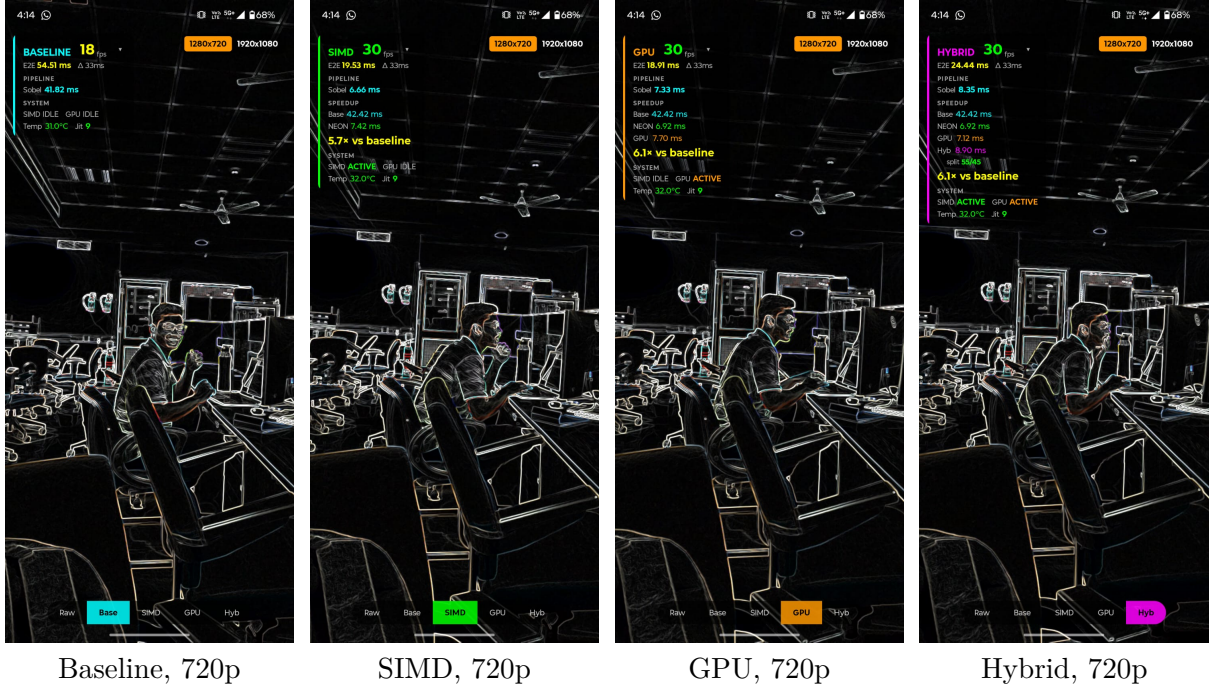


Figure 1: Representative runtime screenshots from the final application. We include only a compact subset to show the main comparison between baseline and the three accelerated modes in a single page.

from the `ImageReader` until the processed bitmap is ready after Sobel processing, bitmap copy, and rotation for portrait display. The **Sobel latency** is measured only around the selected filtering function, so it isolates the cost of the currently active processing mode.

The **speedup** shown on the dashboard is computed from stabilized per-mode averages, not from a single screenshot frame. In the Kotlin code, the app ignores the first **three** frames after a mode switch as warm-up. It then maintains up to **thirty** Sobel-latency samples for each mode and updates a running average for baseline, SIMD, GPU, and hybrid independently. If the average baseline Sobel latency is denoted by T_{base} and the best available accelerated average by

$$T_{best} = \min(T_{simd}, T_{gpu}, T_{hyb}),$$

then the large speedup number shown on the dashboard is

$$Speedup = \frac{T_{base}}{T_{best}}.$$

This is why the yellow “× vs baseline” value in the screenshot is sometimes different from the ratio of the instantaneous Sobel numbers shown in the same frame. The code intentionally compares stable mode averages so that the reported improvement remains fair across mode switches and after resolution changes, where all mode statistics are reset.

The **hybrid split ratio** is computed from the row index where the frame is divided between CPU SIMD and GPU. In the native C++ code, the top part of the frame is

processed by a CPU thread running the NEON Sobel routine, while the bottom part is processed by the GPU compute shader. If the split occurs at row r in a frame of height H , then the dashboard shows approximately

$$CPUshare = \frac{r}{H}, \quad GPUshare = 1 - \frac{r}{H}.$$

The app updates this split adaptively using measured *per-row* processing cost on both sides. The native code stores the measured CPU-half time and GPU-half time, converts them into cost per row, smooths them over time, and then selects the next split so that both sides are expected to finish at nearly the same time. The smoothing factor is increased when the observed end-to-end latency is high or when the temperature proxy crosses roughly **40.5°C**, so hybrid balancing reacts more quickly under stress. The implementation also slightly shifts work away from the GPU in that thermal region.

Two other dashboard values are also computed in code and are worth explaining briefly. The **jitter count** is increased only when the current end-to-end latency is greater than twice a rolling average of previous end-to-end latencies. The **SIMD** and **GPU** activity indicators are not inferred indirectly from timings; they are tied directly to the active execution mode, so in hybrid mode both indicators are shown as active simultaneously.

The code also explains some of the engineering decisions behind the measured results. The NEON Sobel path processes **sixteen pixels per vector iteration** and uses explicit prefetching to reduce cache stalls. The GPU path uses a headless EGL context, texture-backed input, and a two-slot readback pipeline with synchronization fences so that the CPU can read the previous GPU result while the next dispatch is running. Before normal GPU use, the application benchmarks three compute-shader workgroup variants (8×8 , 16×16 , and 32×4) and keeps the fastest one for subsequent frames. These code-level details are important because the reported speedups are a consequence of the actual implementation, not just of the final screenshots.

15. What This Project Demonstrates

This project demonstrates several important ideas from mobile systems and performance engineering.

First, a mobile application can meaningfully use more than one type of compute resource. CPU scalar execution, CPU SIMD execution, and GPU execution each have different strengths and costs.

Second, acceleration is useful only when measured honestly in the complete pipeline. A faster kernel alone does not guarantee a better system.

Third, observability matters. Without the live dashboard, the project would be much harder to evaluate or explain. The dashboard makes the behavior of the system visible and helps connect performance numbers to actual runtime behavior.

Fourth, hybrid execution is not only a theoretical concept. With proper coordination, both CPU SIMD and GPU can contribute to the same frame-processing workload.

16. Contributions and AI Usage

16.1 Team Contributions

The work was divided equally across all four development phases, with each team member taking ownership of specific stages within each phase.

In Phase 1 (Camera Pipeline and Baseline System), Nitish Kumar set up the Android project and the NDK toolchain, integrated the Camera2 API for continuous frame capture on a background thread, and implemented the YUV-to-RGBA conversion in native C++. Salil Gujar wrote the scalar Sobel edge detection filter including border handling, configured the SurfaceView rendering path, and designed the initial live performance dashboard.

In Phase 2 (SIMD Acceleration), Nitish developed the ARM NEON intrinsics warm-up experiment and wrote the NEON-vectorized Sobel filter that processes sixteen pixel positions at a time using 128-bit vector registers. Salil built the correctness verification framework that compares the SIMD output against the scalar baseline within a tight numerical tolerance, implemented the runtime mode-switching UI, and extended the dashboard with SIMD activity indicators and the speedup ratio display.

In Phase 3 (GPU Acceleration and Hybrid Mode), Nitish set up the headless EGL compute context, wrote the OpenGL ES compute shader for GPU Sobel processing with texture-based input, and implemented the hybrid frame-split execution logic. Salil handled GPU correctness verification, performed sustained load and stress testing across all four mode transitions, and updated the dashboard with GPU and hybrid-specific indicators including the live split-ratio display.

In Phase 4 (End-to-End Optimization and Finalization), Nitish carried out pipeline latency profiling using millisecond timestamps, drove memory and buffer optimization to eliminate garbage-collection pauses, and tuned the NEON filter loops for cache efficiency. Salil worked on GPU shader workgroup-size experimentation and the switch from SSBOS to texture-based access, implemented the dynamic hybrid split-ratio adjustment, and finalized the dashboard layout. The 30-minute final stability run and integration testing were a joint effort by both team members.

16.2 AI Usage

We used Large Language Models(Chatgpt, Gemini) as a support tool throughout this project, treating the AI as a collaborative assistant rather than a primary developer.

We designed the core system architecture, the SIMD acceleration strategy, and the GPU integration approach ourselves. In particular, the idea to implement a hybrid execu-

tion mode by splitting the camera frame horizontally and routing each half concurrently to the CPU SIMD path and the GPU compute path was entirely our own concept. We also designed the adaptive balancing mechanism that dynamically adjusts the split ratio based on measured per-row throughput of each hardware resource so that both sides finish at roughly the same time. The AI did not suggest or contribute to this hybrid splitting strategy.

For the NEON vectorization path, which we understood well due to our background in CPU-level optimization, the AI was used mainly for debugging, clarifying obscure compiler error messages, and performing static analysis to confirm that our logic was correct.

For the GPU and hybrid modes, since we had less prior experience with the OpenGL ES compute API, the AI helped generate boilerplate code for EGL setup, shader compilation, and buffer management. We provided the high-level design and the hybrid strategy; the AI helped translate those ideas into working GLSL and C++ code. This allowed us to focus on system-level integration and adaptive balancing rather than low-level API mechanics that were new to us.

17. Conclusion

As a group of two students, we approached this project as a complete end-to-end mobile systems problem rather than a narrow optimization exercise. We began with a correct scalar baseline, then expanded the system step by step through SIMD acceleration, GPU acceleration, and finally hybrid execution. At each stage, we focused on correctness, runtime behavior, and live measurement.

The final result is a real-time Android application that captures camera frames, processes them continuously on the device, displays the transformed output live, and makes the use of mobile hardware acceleration visible through a dashboard. The system now supports multiple execution modes at runtime, reports meaningful performance information, and demonstrates how heterogeneous mobile resources can be used for a real-time workload.

More than just building a faster Sobel filter, the project helped us understand how camera pipelines, native code, SIMD execution, GPU compute, synchronization, and measurement all interact in a real mobile system. The most important point is that our work now matches the assignment in practical terms: live camera input, accelerated frame processing, continuous on-screen display, four runtime modes, a visible dashboard, and clear improvement over baseline. That is the main value of the work we have done.